



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Computational Intelligence

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft

Klassifikation

k-Nearest Neighbour

Was erwartet Dich in diesem Kapitel?

k-Nearest Neighbour - Einführung Algorithmus

- diskrete Funktionen
 - reellwertige Funktionen
- ## verfeinerter Algorithmus
- Shepard's method



Bildquelle: <https://www.lesen.net/wp-content/uploads/2014/01/zettelkasten-347x195.jpg>

Klassifikation mittels k-Nearest Neighbour

k-Nearest Neighbour ist ein Verfahren, das im Data Mining für die Anwendungsklasse Klassifikation verwendet werden kann.

- Vorhersage aus Trainingsdaten → *Instanzbasiertes Verfahren*
- setzt Abstandsmaß voraus



ACHTUNG: viele Implementierungen verlangen reelwertige Attribute, das ist aber **nicht zwingend** nötig!!!
(auch nominale und ordinale Attribute möglich, das Abstandsmaß muss passen)

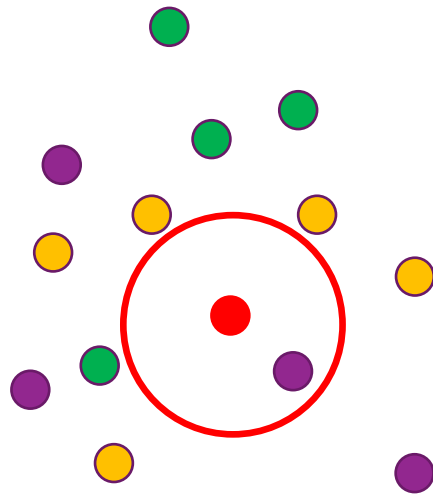
Wir brauchen ...

... gespeicherte Beispielobjekte

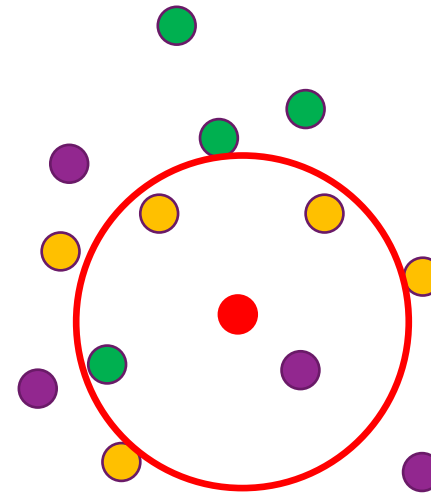
Neue Objekte werden aufgrund der Ähnlichkeit ihrer beschreibenden Attribute zu den gespeicherten Objekten klassifiziert:

- Wahl von **k** Objekten, die dem neuen Objekt am nächsten sind,
- die unter den k nächsten Nachbarn häufigste Klasse bestimmt die Vorhersage des neuen Objekts

Beispiel mit euklidischer Distanz



k=1
Klasse Klammer



k=4
Klasse Nagel

Legende:

● Schraube

● Nagel

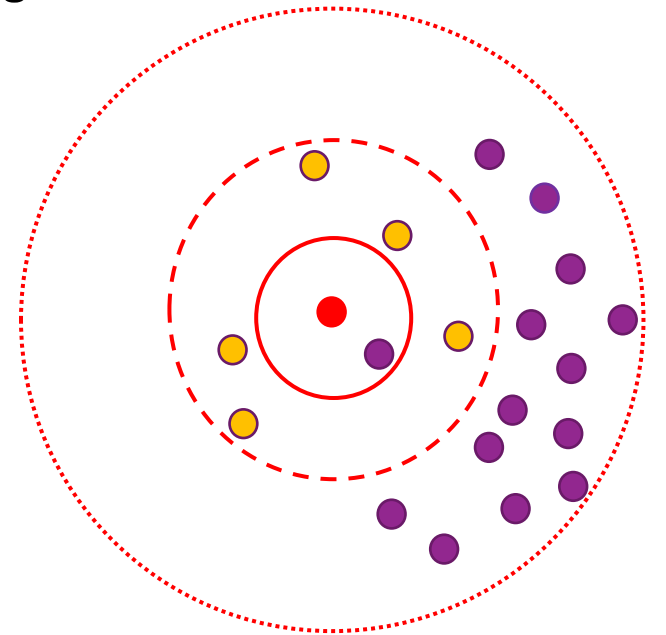
● Klammer

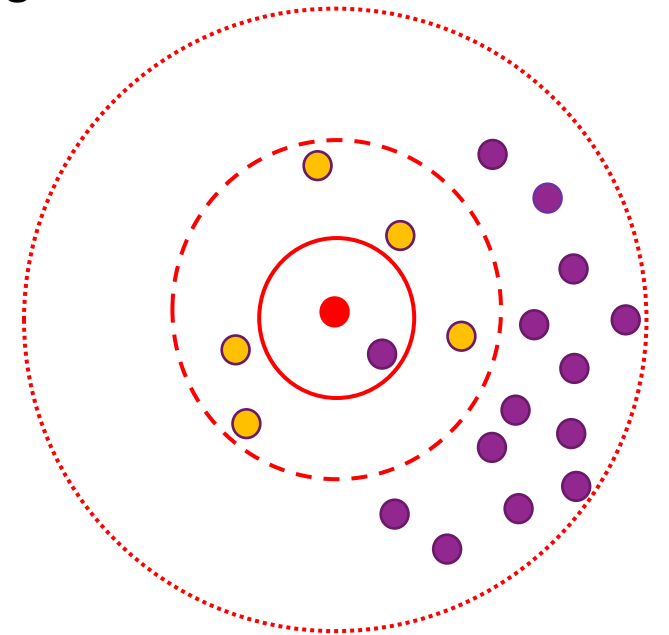
● zu klassifizierender
Datensatz

Erkenntnisse - Entscheidungsmenge

Für das Resultat kann also die (un)geschickte Wahl von k maßgeblich sein.

- „zu kleines“ k :
hohe Sensitivität gegen über Ausreißern
- „zu großes“ k :
viele Objekte aus anderen Clustern (Klassen) in der Entscheidungsmenge
- mittleres k :
höchste Klassifikationsgüte, oft $1 \ll k < 10$

- 
- Entscheidungsmenge für $k=1$
 - Entscheidungsmenge für $k=19$
 - Entscheidungsmenge für $k=6$



Deshalb:

mehrere Varianten mit unterschiedlichen k durchrechnen und vorhergesagte Klassen vergleichen

Erkenntnisse - Entscheidungsregel

- Die Komplexität des Verfahrens wird weniger durch den Lernschritt, als vielmehr durch den Klassifikationsschritt bestimmt.
- Da aber keine komplette und kompakte Beschreibung der zu erlernenden Funktion generiert werden muss, kann sich die unter Umständen teure Berechnung der Ähnlichkeit auf einen als relevant ermittelten Suchbereich beschränken.
- Diese zentralen Merkmale machen das k-Nearest Neighbour-Lernen zu einem engen Verwandten beispielsweise des **fallbasierten Schließens** und der **lokal gewichteten Regression**.



Die Annahme, dass geringer Abstand zweier Punkte die gleiche Klasse bestimmt, muss kritisch hinterfragt werden!

Ist hier aber Voraussetzung ...

Standard-Algorithmus

Wie bestimmen wir nun aus den Klassen der Entscheidungsmenge die Klasse des zu klassifizierenden Objektes?

Simple Idee:

Sei f der Klassifikator.

- für jeden Trainingsbeispiel-Datensatz x :
speichere $(x, f(x))$ in einer Trainingsbeispielmenge
- ermittle für einen neuen Datensatz x' den Funktionswert $f(x')$
- wähle die Mehrheitsklasse der Entscheidungsmenge

Die Funktion f berechnet den Wert, der unter den Klassifikationswerten der k am nächsten gelegenen Trainingsbeispiele am häufigsten vorkommt, d.h. x' wird der Klasse zugeordnet, die unter den k ähnlichsten Beispielen am meisten vertreten ist.
Diese wird dann für den neuen Datensatz vorhergesagt.

Diskrete Funktionen

Seien $V := \{v_1, v_2, \dots, v_m\}$ eine endliche Menge (Werte, die das Zielattribut annehmen kann) und $f: \mathcal{R}^n \rightarrow V$ die zu erlernende Funktion und y der Vektor des zu klassifizierenden Beispiels.

- Für alle $x_i \in \text{Trainingsbeispielmenge}$ berechne die Ähnlichkeit zu y .
- Wähle die k Beispiele $x_1, x_2, \dots, x_k$ aus, die zu y am ähnlichsten sind.
- Gebe aus:

$$\text{klasse}(y) := \max_{v \in V} \sum_{p=1}^k \delta(v, f(x_p)) \text{ mit } \delta(a, b) := \begin{cases} 1 & \text{falls } a = b \\ 0 & \text{sonst} \end{cases}$$

Beispiel (1) - Trainingsbeispielmenge

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Einkommen
1	59	ja	ja	ja	hoch
2	55	ja	nein	nein	gering
3	40	nein	nein	nein	gering
4	37	ja	ja	ja	hoch
5	26	nein	nein	nein	gering
6	24	ja	nein	nein	mittel
7	22	ja	ja	ja	mittel
8	53	nein	ja	nein	hoch

Vorherzusagendes Attribut: **Einkommen**, also Gehaltsgruppe *hoch, mittel* oder *gering*

Benötigt werden: k und Distanzfunktion

Beispiel (2) gesucht ist Gehaltsgruppe für

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Einkommen
0	26	ja	nein	ja	?

Wir wählen $k=2$ und die Hamming-Distanz.

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Abstand	Einkommen
1	59	ja	ja	ja		hoch
2	55	ja	nein	nein		gering
3	40	nein	nein	nein		gering
4	37	ja	ja	ja		hoch
5	26	nein	nein	nein		gering
6	24	ja	nein	nein		mittel
7	22	ja	ja	ja		mittel
8	53	nein	ja	nein		hoch

Vorhersage Gehaltsgruppe: ???

2xhoch, 2xmittel und 2xgering und welche davon sind jetzt die 2 **nächsten Nachbarn**???



Was ist zu tun, wenn wir weiterhin Hamming-Distanz benutzen wollen?

Altersangabe muss in Kategorien gebracht werden!

Beispielsweise

- 0 – 29 → *Jung*
- 30 – 49 → *Mittel*
- 50 – → *Alt*

Beispiel (3) gesucht ist Gehaltsgruppe für

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Einkommen
0	jung	ja	nein	ja	?

Wir wählen $k=2$ und die Hamming-Distanz.

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Abstand	Einkommen
1	alt	ja	ja	ja		hoch
2	alt	ja	nein	nein		gering
3	mittel	nein	nein	nein		gering
4	mittel	ja	ja	ja		hoch
5	jung	nein	nein	nein		gering
6	jung	ja	nein	nein		mittel
7	jung	ja	ja	ja		mittel
8	alt	nein	ja	nein		hoch

Vorhersage Gehaltsgruppe: *mittel*

Andere Alternative: Euklidische Distanz



Was ist zu tun, wenn wir die Euklidische Distanz benutzen wollen?

- Alle Werte numerisch!
Ja=1, Nein=0
- Altersangaben normalisieren!

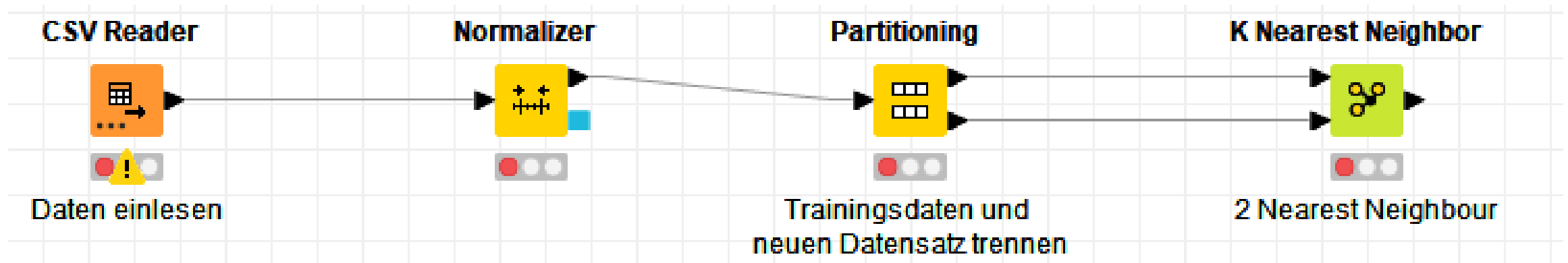
Beispiel (4) gesucht ist Gehaltsgruppe für

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Einkommen
0	0,11	1	0	1	?

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Abstand	Einkommen
1	1,0	1	1	1		hoch
2	0,89	1	0	0		gering
3	0,49	0	0	0		gering
4	0,41	1	1	1		hoch
5	0,11	0	0	0		gering
6	0,05	1	0	0		mittel
7	0,0	1	1	1		mittel
8	0,84	0	1	0		hoch

Vorhersage Gehaltsgruppe: *mittel*

KNIME und k-Nearest Neighbour



Reellwertige Funktionen

Seien $V := \{v_1, v_2, \dots, v_m\}$ eine endliche Menge (Werte, die das Zielattribut annehmen kann) und $f: \mathcal{R}^n \rightarrow \mathcal{R}$ die zu erlernende Funktion und y der Vektor des zu klassifizierenden Beispiels.

- Für alle $x_i \in \text{Trainingsbeispielmenge}$ berechne die Ähnlichkeit zu y .
- Wähle die k Beispiele $x_1, x_2, \dots, x_k$ aus, die zu y am ähnlichsten sind.
- Gebe aus:

$$f'(y) := \frac{\sum_{p=1}^k f(x_p)}{k}$$



Eigentlich ist das nun keine Klassifikation mehr, sondern eine numerische Vorhersage!

Beispiel (5)

Wir tragen nun also Zahlen in die Spalte Einkommen ein:

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Abstand	Einkommen
1	1,0	1	1	1		4050
2	0,89	1	0	0		950
3	0,49	0	0	0		1005
4	0,41	1	1	1		3890
5	0,11	0	0	0		800
6	0,05	1	0	0		2300
7	0,0	1	1	1		2700
8	0,84	0	1	0		3780

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Einkommen	
0	0,11	1	0	1	?	



Ändert sich etwas bei Berechnung der 2 ähnlichsten Datensätze?

Beispiel (6)

Nein, denn der Abstandsberechnung hat nichts mit dem Zielattribut zu tun.

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Abstand	Einkommen
1	1,0	1	1	1	1,34	4050
2	0,89	1	0	0	1,27	950
3	0,49	0	0	0	1,464	1005
4	0,41	1	1	1	1,043	3890
5	0,11	0	0	0	1,414	800
6	0,05	1	0	0	1,001	2300
7	0,0	1	1	1	1,006	2700
8	0,84	0	1	0	1,879	3780

Nr	Alter	verheiratet	Eigenheim	Akademiker*in	Einkommen
0	0,11	1	0	1	2500

Die beiden ähnlichsten Datensätze sind weiterhin 6 und 7! $(2300+2700) / 2 = 2500$

Verfeinerung des Algorithmus

- ▶ alle der k ausgewählten Beispiele gehen zu **gleichen** Teilen in Ergebnis des Klassifikationsschrittes ein
- ▶ unter der Annahme, dass ein geringer Abstand eine hohe Ähnlichkeit bedeutet, erscheint es sinnvoll, jedem einzelnen Ergebnisvektor entsprechend seinem Abstand zum Anfragevektor ein Gewicht zuzuordnen
- ▶ man könnte/kann nun nicht nur die k nächsten Nachbarn, sondern alle Trainingsbeispiele zur Ermittlung des Ergebnisses verwenden
(solche mit hohem Abstand tragen kaum noch zum Ergebnis bei)
- ▶ höherer Rechenaufwand im Klassifikationsschritt als beim Standardvorgehen

Ideen:

- Interpretiere Häufigkeit einer Klasse in der Entscheidungsmenge als Wahrscheinlichkeit der Klassenzugehörigkeit
 - Maximum-Likelihood-Prinzip: Mehrheitsentscheidung
 - Ggf. Gewichtung
- Gewichtete Entscheidungsregel
- gewichte die Klassen der Entscheidungsmenge
- nach Distanz, meist invers quadriert: $weight(dist) = 1/dist^2$
 - nach Verteilung der Klassen (oft sehr ungleich!) Problem: Klasse mit zu wenig Instanzen ($< k/2$) in der Trainingsmenge bekommt keine Chance, ausgewählt zu werden, selbst bei optimaler Distanzfunktion Klasse A: 95 %, Klasse B 5 %
 - Entscheidungsmenge = {A, A, A, A, B, B, B}
 - Standardregel $\Rightarrow$ A, gewichtete Regel $\Rightarrow$ B

Shepards method – Diskrete Funktionen

Als Gewicht wird das Inverse des Quadrats der Distanz zwischen Trainings- und Anfragevektor gewählt.

Distanzgewichtung verfeinerte Klassifikationsschritt im diskreten Fall:

- Sei $V := \{v_1, v_2, \dots, v_m\}$ eine endliche Menge (Werte, die das Zielattribut annehmen kann)
- Für alle $x_i \in \text{Trainingsbeispielmenge}$ berechne die Ähnlichkeit zu y .
- Wähle die k Beispiele $x_1, x_2, \dots, x_k$ aus, die zu y am ähnlichsten sind.
- Gebe aus:

$$\text{klasse}(y) := \begin{cases} f(x_i) & \text{falls } y = x_i \text{ für ein } i \\ \max_{v \in V} \sum_{p=1}^k w_p * \delta(v, f(x_p)) & \text{sonst} \end{cases}$$

$$\text{mit } \delta := \begin{cases} 1 & \text{falls } a = b \\ 0 & \text{sonst} \end{cases} \quad \text{und} \quad w_p := \frac{1}{\text{dist}(y, x_p)^2}$$

Shepards method – reellwertige Funktionen

Als Gewicht wird das Inverse des Quadrats der Distanz zwischen Trainings- und Anfragevektor gewählt.

Distanzgewichtung verfeinerte Klassifikationsschritt im reellwertigen Fall:

Seien $V := \{v_1, v_2, \dots, v_m\}$ eine endliche Menge (Werte, die das Zielattribut annehmen kann) und y der Vektor des zu klassifizierenden Beispiels.

- Für alle $x_i \in \text{Trainingsbeispielmenge}$ berechne die Ähnlichkeit zu y .
- Wähle die k Beispiele $x_1, x_2, \dots, x_k$ aus, die zu y am ähnlichsten sind.
- Gebe aus:

$$f'(y) := \begin{cases} f(x_i) & \text{falls } y = x_i \text{ für ein } i \\ \frac{\sum_{p=1}^k w_p f(x_p)}{k} & \text{sonst} \end{cases}$$

mit $w_p := \frac{1}{\text{dist}(y, x_p)^2}$

Anmerkungen

- ▶ Mit $k \gg 1$ arbeitet der kNN-Algorithmus meist auch bei **verrauschten** (reellwertig mit Streuung) Trainingsdaten sehr gut.
- ▶ **Alle Attribute** werden in die Berechnung einbezogen (im Gegensatz zu anderen Verfahren wie Entscheidungsbaum).
 - ▶ bedeutet, der Algorithmus verliert stark **an Zuverlässigkeit**, wenn einige der Attribute für die Bestimmung der Ähnlichkeit irrelevant sind.
 - ▶ Durch gewichtete Attribute lässt sich dieses Problem abmildern.
- ▶ Trainingsdaten sollten den Lösungsraum möglichst gleichmäßig aufspannen.

Klassifizierungsbeispiel (1) - Trainingsmenge

Tag	Outlook	Temperature	Humidity	Windy?	Sailing?
1	sunny	hot	high	true	no
2	sunny	hot	high	false	no
3	overcast	hot	high	true	yes
4	rainy	mild	high	true	yes
5	rainy	cool	normal	true	yes
6	rainy	cool	normal	false	no
7	overcast	cool	normal	false	yes
8	sunny	mild	high	true	no
9	sunny	cool	normal	true	yes
10	rainy	mild	normal	true	yes
11	sunny	mild	normal	false	yes
12	overcast	mild	high	false	yes
13	overcast	hot	normal	true	yes
14	rainy	mild	high	false	no

Klassifizierungsbeispiel (2)

Welches Abstandsmaß wählen?

Zu klassifizieren sind

Tag	Outlook	Temperature	Humidity	Windy?	Sailing?
X	sunny	mild	normal	false	?
Y	sunny	mild	normal	true	?
Z	rainy	hot	normal	false	?

Gern mit verschiedenen k experimentieren!